

Mercado Exclusivo

Marketplace de Compra y Venta

Trabajo Práctico Obligatorio — Entrega 1

Aplicaciones Interactivas | UADE | 1er Cuatrimestre 2026

Grupo 4 - Alumnos:

- ACEVEDO ARECO, PABLO
- ANDREUS, ANDREA
- CAPONE, FABRICIO
- MORELLO FLORES, MARIA LAURA
- SALVATIERRA, MARTINA



UADE

UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

1. Descripción del Proyecto

Mercado Exclusivo es un marketplace de compra y venta de productos de alta gama, desarrollado en Java con Spring y MySQL, similar en concepto a sistemas como MercadoLibre. La plataforma está orientada a un segmento exclusivo, ofreciendo artículos de lujo, edición limitada y colección, para una audiencia selecta. Permite a los usuarios registrarse, publicar productos para vender y realizar compras, todo dentro de un mismo sistema con autenticación segura, mediante JWT y control de acceso por roles.

Link GitHub: <https://github.com/pacevedoareco/apps-interactivas>

2. Tecnologías utilizadas

Tecnología	Detalle
Spring / Spring boot	Framework y template de Proyecto
Spring Data JPA / Hibernate	ORM y persistencia
Spring Security + JWT	Autenticación y autorización
MySQL	Base de datos relacional
Lombok	Reducción de boilerplate
Maven	Gestión de dependencias
Swagger/Postman	Prueba de endpoints

3. Arquitectura del Proyecto

El proyecto sigue una arquitectura en capas, separando claramente las responsabilidades:

- **Presentación (Controllers):** Recibe los requests HTTP y devuelve las respuestas.
- **Lógica de Negocio (Services):** Contiene la lógica del negocio, validaciones y conversiones entre entidades y DTOs.
- **Acceso a Datos (Repositories):** Extiende JpaRepository para operaciones CRUD sobre la base de datos.
- **Dominio (Models + DTOs):** Entidades JPA mapeadas a tablas y DTOs para transferir datos sin exponer la entidad.
- **Seguridad (Security):** Spring Security con JWT para autenticación y autorización basada en roles.

- **Manejo de Excepciones:** @ControllerAdvice con excepciones personalizadas para respuestas HTTP consistentes.

4. Decisiones de Diseño

4.1 Generales

Baja lógica en lugar de eliminación física

Ningún registro se elimina físicamente de la base de datos. Cada módulo tiene su propio mecanismo: Usuario usa activo=false, Producto usa CondicionPublicacion.ELIMINADA y Pedido usa EstadoPedido.CANCELADO. Esto preserva el historial de transacciones y datos relacionados.

DTOs separados para entrada y salida

Se usan DTOs distintos para entrada (RequestDTO) y salida (ResponseDTO) en todos los módulos. Esto evita exponer datos sensibles como la contraseña y permite controlar exactamente qué campos acepta y devuelve cada endpoint.

Excepciones personalizadas por módulo

Cada módulo tiene sus propias excepciones que extienden RuntimeException. Un GlobalExceptionHandler centralizado las captura y devuelve respuestas HTTP consistentes al cliente.

FK en las tablas hijas

En todas las relaciones la FK queda en la tabla hija apuntando a la tabla padre. En OneToMany la FK está en el lado Many. En OneToOne la FK está en la entidad relacionada.

FetchType.LAZY en todas las relaciones

Todas las relaciones JPA usan LAZY loading. Los datos relacionados solo se cargan cuando se solicitan explícitamente con el getter, evitando consultas innecesarias a la base de datos.

4.2 Usuario

Un solo Usuario para comprador y vendedor

Un mismo usuario puede tanto comprar como vender. El rol (USER/ADMIN) no determina si puede comprar o vender, sino el nivel de acceso a la plataforma.

Role enum (USER, ADMIN)

El rol del usuario se maneja con un enum para garantizar consistencia y evitar valores inválidos. USER tiene acceso a las funcionalidades del marketplace, ADMIN gestiona la plataforma.

Dirección como entidad separada con OneToOne

La dirección del usuario es una entidad separada en su propia tabla con FK usuario_id en la tabla dirección. Esto evita tener un campo de texto largo en la tabla usuarios y permite estructurar correctamente los datos de cada dirección asociada a un usuario.

4.3 Producto

EstadoProducto enum (NUEVO, USADO, EDICION_LIMITADA)

El estado del artículo se clasifica con un enum para garantizar valores válidos. Por defecto se asigna NUEVO si no se informa.

CondicionPublicacion enum (ACTIVA, PAUSADA, ELIMINADA)

El ciclo de vida de la publicación se maneja con un enum. ELIMINADA es el mecanismo de baja lógica del módulo. Por defecto se asigna ACTIVA si no se informa.

ManyToMany con Categoria mediante tabla intermedia

Un producto puede pertenecer a múltiples categorías y una categoría puede tener múltiples productos. La relación se resuelve con la tabla intermedia producto_categoria.

4.4 Pedido

ItemPedido como entidad separada

Cada producto dentro de un pedido es una entidad ItemPedido con su propia tabla, almacenando cantidad, precio unitario y subtotal de forma independiente.

Precio unitario guardado al momento de la compra

El precio se guarda en ItemPedido al crear el pedido, no se referencia el precio actual del producto. Esto garantiza que el historial siempre refleje el precio real que pagó el usuario, independientemente de cambios futuros.

Cálculo automático con @PrePersist

ItemPedido calcula el subtotal automáticamente antes de persistirse. Pedido asigna la fechaCreacion y el estado inicial PENDIENTE de la misma forma, evitando que el cliente envíe estos datos.

EstadoPedido enum (PENDIENTE, CANCELADO)

El ciclo de vida del pedido se maneja con un enum. CANCELADO es el mecanismo de baja lógica del módulo.

4.5 Carrito

EstadoCarrito enum (ACTIVO, CHECKOUT_REALIZADO, ABANDONADO)

El ciclo de vida del carrito se maneja con un enum. CHECKOUT_REALIZADO indica que el carrito fue procesado y está a la espera de una nueva compra.

CarritoItem como entidad separada

Cada producto dentro del carrito es una entidad CarritoItem con su propia tabla, almacenando cantidad, precio unitario y subtotal de forma independiente.

Total recalculado automaticamente

El total del carrito se recalcula automáticamente con @PreUpdate cada vez que se modifica, agrega o elimina un ítem.

Checkout con validación completa de stock

El checkout valida el stock de todos los items antes de confirmar. Si alguno falla, rechaza toda la operación y devuelve los errores específicos por producto. Si todo es válido, descuenta el stock, genera el pedido y vacía el carrito.

5. Entidades y Relaciones JPA

5.1 Usuario

Entidad central del marketplace. Un usuario puede actuar tanto como comprador como vendedor.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
nombre	String	Nombre del usuario (NOT NULL)
apellido	String	Apellido del usuario (NOT NULL)
email	String	Email único, actúa como username (NOT NULL, UNIQUE)
password	String	Contraseña encriptada con BCrypt (NOT NULL)
telefono	String	Teléfono de contacto (opcional)
activo	boolean	Estado de la cuenta. false = suspendida
role	Enum	Rol: USER o ADMIN (NOT NULL)
fechaNacimiento	Date	Fecha de nacimiento (opcional)

Relaciones de Usuario

Atributo	Entidad	Tipo	Descripción
direccion	Direccion	OneToOne	Dirección de envío del usuario
productosPublicados	Producto	OneToMany	Productos publicados como vendedor
pedidos	Pedido	OneToMany	Historial de compras realizadas

carrito	Carrito	OneToOne	Carrito de compras activo
---------	---------	----------	---------------------------

5.2 Direccion

Direccion de envio asociada a un usuario. Relacion OneToOne con FK usuario_id en tabla direccion.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
calle	String	Nombre de la calle
numero	String	Número de puerta
piso	String	Piso (opcional)
departamento	String	Departamento (opcional)
ciudad	String	Ciudad
provincia	String	Provincia
codigoPostal	String	Código postal
pais	String	País
usuario_id	Long (FK)	FK que referencia a la tabla usuarios

5.3 Producto

Representa una publicación de venta en el marketplace. Cada producto pertenece a un vendedor (Usuario) y puede pertenecer a múltiples categorías (ManyToMany).

Atributo	Tipo	Descripción
idProducto	Long (PK)	Identificador único autoincremental
nombre	String	Nombre del producto
descripcion	String	Descripción del producto
precio	Double	Precio del producto
stock	Integer	Stock disponible
marca	String	Marca del producto (opcional)

estadoProducto	Enum	NUEVO, USADO, EDICION_LIMITADA
condicionPublicacion	Enum	ACTIVA, PAUSADA, ELIMINADA
fechaPublicacion	LocalDate	Fecha de publicación (se asigna automáticamente)
vendedor_id	Long (FK)	FK que referencia al usuario vendedor (NOT NULL)

Relaciones de Producto

Atributo	Entidad	Tipo	Descripción
vendedor	Usuario	ManyToOne	Cada producto tiene un único vendedor
categorias	Categoria	ManyToMany	Un producto puede pertenecer a múltiples categorías

5.4 Categoria

Clasifica los productos del marketplace. Relación ManyToMany con Producto a través de la tabla intermedia producto_categoria.

Atributo	Tipo	Descripción
idCategoria	Long (PK)	Identificador único autoincremental
nombre	String	Nombre de la categoría
descripcion	String	Descripción de la categoría

Relaciones de Categoria

Atributo	Entidad	Tipo	Descripción
productos	Producto	ManyToMany	Una categoría puede tener múltiples productos (lado inverso)

5.5 Pedido

Representa una compra realizada. Contiene una lista de ItemPedido con los productos comprados, cantidades y precios.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
fechaCreacion	LocalDateTime	Fecha de creación (se asigna automáticamente con @PrePersist)
estado	Enum	PENDIENTE, CANCELADO
total	Double	Total calculado automáticamente
usuario_id	Long (FK)	FK que referencia al usuario comprador (NOT NULL)

Relaciones de Pedido

Atributo	Entidad	Tipo	Descripción
usuario	Usuario	ManyToOne	Cada pedido pertenece a un único usuario comprador
items	ItemPedido	OneToMany	Un pedido contiene uno o más items

ItemPedido

Entidad que representa cada producto dentro de un pedido, con su cantidad y precio unitario al momento de la compra.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
cantidad	Integer	Cantidad del producto
precioUnitario	Double	Precio del producto al momento de la compra
subtotal	Double	Calculado automáticamente con @PrePersist
producto_id	Long (FK)	FK que referencia al producto (NOT NULL)
pedido_id	Long (FK)	FK que referencia al pedido (NOT NULL)

Relaciones de ItemPedido

Atributo	Entidad	Tipo	Descripción
producto	Producto	ManyToOne	Cada ítem referencia un producto específico
pedido	Pedido	ManyToOne	Cada ítem pertenece a un pedido

5.6 Carrito

Representa el carrito de compras activo del usuario. Se crea automáticamente al primer acceso y se asocia de forma única a cada usuario.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
fechaCreacion	LocalDateTime	Asignada automáticamente con @PrePersist
fechaActualizacion	LocalDateTime	Actualizada automáticamente con @PreUpdate
estado	Enum	ACTIVO, CHECKOUT_REALIZADO, ABANDONADO
total	Double	Recalculado automáticamente al modificar ítems
usuario_id	Long (FK)	FK que referencia al usuario dueño del carrito (UNIQUE)

Relaciones de Carrito

Atributo	Entidad	Tipo	Descripción
usuario	Usuario	OneToOne	Cada carrito pertenece a un único usuario
items	CarritoItem	OneToMany	Un carrito contiene uno o más ítems

CarritoItem

Entidad que representa cada producto dentro del carrito, con su cantidad y precio unitario al momento de agregarlo.

Atributo	Tipo	Descripción
id	Long (PK)	Identificador único autoincremental
cantidad	Integer	Cantidad del producto
precioUnitario	Double	Precio al momento de agregar al carrito
subtotal	Double	Calculado automaticamente con @PrePersist/@PreUpdate
carrito_id	Long (FK)	FK que referencia al carrito
producto_id	Long (FK)	FK que referencia al producto

Relaciones de CarritoItem

Atributo	Entidad	Tipo	Descripción
carrito	Carrito	ManyToOne	Cada ítem pertenece a un carrito
producto	Producto	ManyToOne	Cada ítem referencia un producto específico

6. Endpoints

6.1 Autenticación

Método	Endpoint	Descripción	Acceso
POST	/api/auth/register	Registrar usuario nuevo	Público
POST	/api/auth/login	Login y obtener token JWT	Público

6.2 Usuario — Perfil

Método	Endpoint	Descripción	Acceso
GET	/api/usuarios/{id}	Ver perfil de un usuario	Público
PUT	/api/usuarios/{id}	Editar perfil propio	Autenticado
PUT	/api/usuarios/{id}/desactivar	Desactivar cuenta propia	Autenticado
GET	/api/usuarios/{id}/productos	Ver productos publicados	Público

GET	/api/usuarios/{id}/pedidos	Ver pedidos realizados	Autenticado
GET	/api/usuarios/{id}/carrito	Ver carrito activo (TODO)	Autenticado

6.3 Usuario — Admin

Método	Endpoint	Descripción	Acceso
GET	/api/admin/usuarios	Listar todos los usuarios	ADMIN
GET	/api/admin/usuarios/{id}	Ver perfil de cualquier usuario	ADMIN
PUT	/api/admin/usuarios/{id}/desactivar	Desactivar cuenta de usuario	ADMIN
PUT	/api/admin/usuarios/{id}/activar	Reactivar cuenta de usuario	ADMIN

6.4 Producto

Metodo	Endpoint	Descripción	Acceso
GET	/api/productos	Listar todos los productos	Público
GET	/api/productos/{id}	Ver detalle de un producto	Público
POST	/api/productos	Crear un producto	Autenticado
PUT	/api/productos/{id}	Modificar un producto	Autenticado
DELETE	/api/productos/{id}	Eliminar un producto	Autenticado

6.5 Filtros de Producto: A implementar en próxima entrega

Todos son parámetros opcionales del endpoint GET /api/productos:

Los filtros definidos forman parte del diseño funcional del sistema, pero no fueron implementados en esta primera etapa, ya que se priorizó la construcción de la arquitectura base, persistencia de datos y endpoints CRUD.

Se prevé su implementación en una segunda iteración del proyecto como mejora evolutiva.

Parametro	Ejemplo	Descripción
nombre	?nombre=zapatillas	Buscar por nombre
categoria	?categoria=1	Filtrar por categoría

precioMin + precioMax	?precioMin=100&precioMax=500	Rango de precio
vendedor	?vendedor=1	Productos de un vendedor
estado	?estado=NUEVO	Por estado del producto
condicion	?condicion=ACTIVA	Por condición de publicación
stock	?stock=true	Solo con stock disponible
ordenarPor	?ordenarPor=precio&direccion=ASC	Ordenar resultados

6.6 Pedido

Metodo	Endpoint	Descripción	Acceso
GET	/api/pedidos	Listar todos los pedidos	Autenticado
GET	/api/pedidos/{id}	Ver detalle de un pedido	Autenticado
POST	/api/pedidos	Crear un pedido	Autenticado
PUT	/api/pedidos/{id}/cancelar	Cancelar un pedido	Autenticado

6.7 Carrito (TO-DO)

Metodo	Endpoint	Descripción	Acceso
GET	/api/carrito	Ver carrito del usuario autenticado	Autenticado
DELETE	/api/carrito	Vaciar el carrito	Autenticado
POST	/api/carrito/items	Agregar producto al carrito	Autenticado
PUT	/api/carrito/items/{itemId}	Modificar cantidad de un ítem	Autenticado
DELETE	/api/carrito/items/{itemId}	Eliminar un ítem del carrito	Autenticado
GET	/api/carrito/resumen	Ver resumen con validación de stock	Autenticado
POST	/api/carrito/checkout	Confirmar compra y generar pedido	Autenticado

7. DTOs Implementados

Se utilizan DTOs (Data Transfer Objects) para separar la capa de presentación de la capa de dominio, evitando exponer datos sensibles y controlando exactamente qué información entra y sale de cada endpoint.

7.1 DTOs de Usuario

DTO	Uso	Campos principales
RegisterRequest	POST /auth/register (entrada)	nombre, apellido, email, password, telefono, fechaNacimiento, direccion
LoginRequest	POST /auth/login (entrada)	email, password
UsuarioResponseDTO	GET/PUT (salida)	id, nombre, apellido, email, telefono, activo, role, direccion
UsuarioUpdateDTO	PUT /api/usuarios/{id} (entrada)	nombre, apellido, telefono, fechaNacimiento, direccion
DireccionDTO	Dentro de otros DTOs	calle, numero, piso, departamento, ciudad, provincia, codigoPostal, pais

7.2 DTOs de Producto

DTO	Uso	Campos principales
ProductoRequestDTO	POST/PUT (entrada)	nombre, descripcion, precio, stock, marca, estadoProducto, condicionPublicacion, vendedorId, categoriasIds
ProductoResponseDTO	GET/POST/PUT (salida)	idProducto, nombre, descripcion, precio, stock, marca, estadoProducto, condicionPublicacion, fechaPublicacion, vendedorId, categorias
CategoriaRequestDTO	POST/PUT categoria (entrada)	nombre, descripcion
CategoriaResponseDTO	GET categoria (salida)	idCategoria, nombre, descripcion

7.3 DTOs de Pedido

DTO	Uso	Campos principales
CrearPedidoRequestDTO	POST /api/pedidos (entrada)	usuarioid, items (lista de CrearItemPedidoDTO)

CrearItemPedidoDTO	Dentro de CrearPedidoRequestDTO	productold, cantidad
PedidoResponseDTO	GET/POST (salida)	id, fechaCreacion, estado, total, usuariold, items
ItemPedidoResponseDTO	Dentro de PedidoResponseDTO	id, productold, nombreProducto, cantidad, precioUnitario, subtotal

7.4 DTOs de Carrito (TO-DO)

DTO	Uso	Campos principales
AgregarItemRequestDTO	POST /api/carrito/items (entrada)	productold, cantidad
ModificarItemRequest	PUT /api/carrito/items/{id} (entrada)	cantidad
CarritoResponseDTO	GET/POST (salida)	id, total, estado, fechaCreacion, fechaActualizacion, items
CarritoItemResponseDTO	Dentro de CarritoResponseDTO	id, productold, nombreProducto, cantidad, precioUnitario, subtotal, stockDisponible, stockSuficiente
CarritoResumenResponseDTO	GET /api/carrito/resumen (salida)	total, items, stockValido, erroresStock
StockErrorDTO	Dentro de CarritoResumenResponseDTO	itemId, productold, nombreProducto, cantidadSolicitada, stockDisponible

8. Manejo de excepciones

Se implementó un GlobalExceptionHandler con @ControllerAdvice que captura las excepciones lanzadas en los services y devuelve respuestas HTTP consistentes al cliente.

Excepción	HTTP Status	Cuándo se lanza
UsuarioNotFoundException	404 Not Found	No se encuentra un usuario por id o email
EmailAlreadyExistsException	409 Conflict	El email ya está registrado
ProductoNotFoundException	404 Not Found	No se encuentra un producto por id

PrecioNegativoException	400 Bad Request	El precio del producto es negativo
ResourceNotFoundException	404 Not Found	No se encuentra un recurso genérico (vendedor, categoría)
PedidoNotFoundException	404 Not Found	No se encuentra un pedido por id
StockInsuficienteException	400 Bad Request	No hay stock suficiente para crear el pedido
IllegalArgumentException	400 Bad Request	Se pasan argumentos inválidos
CheckoutStockException	409 Conflict	El checkout falla por stock insuficiente en uno o más productos, devuelve los errores específicos
Exception	500 Internal Server Error	Colchón general para errores inesperados

9. Seguridad

La seguridad está implementada con Spring Security y autenticación mediante JWT (JSON Web Token). Cada request autenticado debe incluir el token en el header:

`Authorization: Bearer {token}`

Endpoint	Acceso
/api/auth/**	Público (register, login)
GET /api/productos/**	Público
POST/PUT/DELETE /api/productos/**	Autenticado
GET /api/usuarios/**	Público (ver perfil de vendedor)
PUT /api/usuarios/**	Autenticado
/api/carrito/**	Autenticado
/api/pedidos/**	Autenticado
/api/admin/**	Solo rol ADMIN

10. Cómo ejecutar el proyecto

- **Requisitos previos:** Java 17+, Maven, MySQL (XAMPP recomendado)
- **Base de datos:** Iniciar XAMPP y activar el servicio MySQL. La BD se crea automáticamente con `createDatabaseIfNotExist=true`
- **Configuración:** Verificar `application.properties` con las credenciales correctas de MySQL
- **Ejecutar:** `./mvnw spring-boot:run` desde la raíz del proyecto
- **Swagger UI:** Acceder a <http://localhost:8080/swagger-ui.html> para ver y probar los endpoints.

Credenciales de prueba

Usuario	Email	Password	Rol
Usuario prueba	juan@mail.com	password123	USER
Administrador	admin@mail.com	password123	ADMIN